Refine Actions If Needed: Adaptive Action Chunk Refinement for Robot Manipulation

Hanlin Liu[1,2,*] Zichen Zhou[2,*] Jiacheng Zhang[1] Liuyuan Na[1,3]
Jin Jin[4] Guangyan Chen[2] Yufeng Yue[2,†] Gen Li[5,†] Fangqiang Ding[1,†]

[1]The Hong Kong University of Science and Technology (Guangzhou)
[2]Beijing Institute of Technology [3]Shanghai Jiao Tong University
[4]University of Oxford [5]Nanyang Technological University
[*]Equal contribution. [†]Corresponding authors.

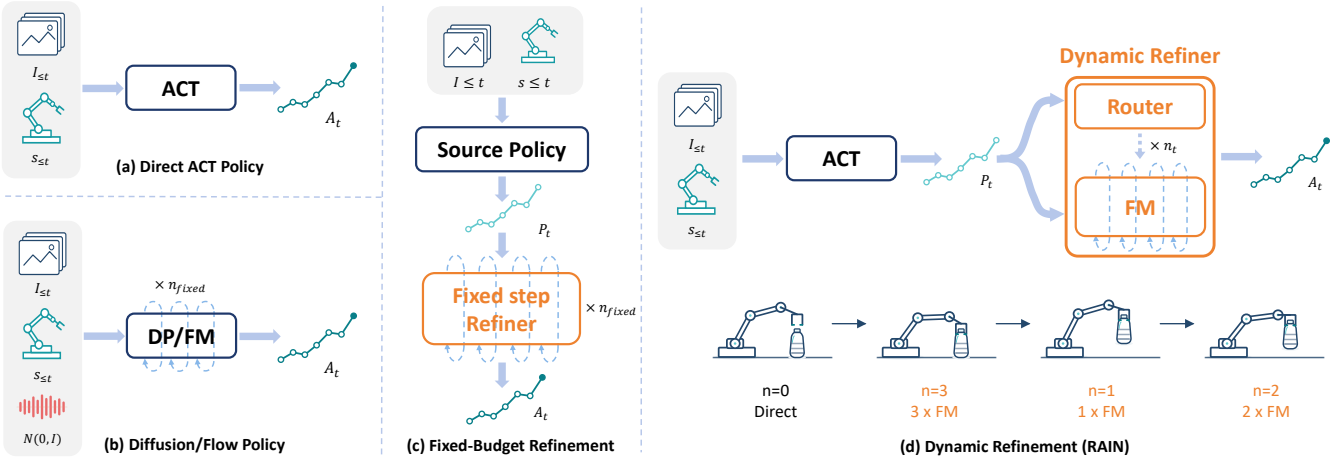


Fig. 1. Comparison of action generation and refinement paradigms. (a) Direct ACT predicts an executable action chunk in a single forward pass. (b) Generative policies such as diffusion policy (DP) and flow matching (FM) generate action chunks through iterative network evaluations. (c) Fixed-budget refinement applies a fixed number of refinement steps to every action proposal. (d) RAIN predicts whether refinement is needed before invoking FM, directly executes proposals that require no refinement, and adaptively allocates additional refinement steps when necessary. The bottle-manipulation sequence illustrates how the computational budget varies across successive policy calls within a single episode.

***Abstract*— Computationally lightweight manipulation policies predict executable actions with low inference latency, but prediction errors can limit task success. Iterative generative policies can achieve higher task success rates, but repeated network evaluations increase inference latency. Refining action proposals from computationally lightweight policies can combine these strengths, but the amount of refinement needed varies across states. We introduce RAIN (Refine Actions If Needed), a framework that predicts action errors at different refinement levels to decide whether and how much to refine each executable proposal, before performing any refinement. We train an ensemble to predict these errors by comparing action proposals and their refined versions with expert actions from offline demonstrations, without requiring explicit labels of refinement needs. Calibration balances uncertainty, computation, and temporal consistency. We evaluate RAIN on eight ManiSkill tasks and four real-robot tasks. In simulation, our policy achieves 66.6% success, approaching the best evaluated diffusion-policy success rate with 5.71× lower policy latency. On the real robot, success increases from 55.0% with Action Chunking with Transformers (ACT) to 72.5%, with comparable average successful completion time. Matched controls show that dynamic routing maintains the strongest observed fixed-refinement success level with 26.6% fewer refiner evaluations.**

Project page: https://rain-policy.github.io/.

I. Introduction

Closed-loop manipulation policies must generate effective actions quickly enough to replan as the environment changes. Diffusion Policy (DP) and flow-matching (FM) policies commonly generate these sequences by transforming an unstructured noise sample into an observation-conditioned action trajectory through repeated network evaluations [1], [2]. This iterative process can improve task success rates, but repeated network evaluations increase inference latency. To reduce inference latency, a computationally lightweight policy can first produce an executable action proposal for a generative model to refine. Instead of generating the entire action sequence from noise, the model can then focus on correcting errors in the proposal.

BRIDGeR and Action-to-Action (A2A) flow matching explore related approaches by replacing noise with policy samples or action-history representations [3], [4]. However, improving the starting point does not by itself determine how much refinement is needed. For an executable proposal, the appropriate choice may range from direct execution to additional correction, depending on the current state. Figure 1

contrasts direct generation, iterative generation, fixed refinement, and our policy with adaptive refinement. The problem is therefore to determine whether the expected improvement over direct execution justifies additional computation, before invoking the refiner.

Making this decision before refinement presents two challenges. First, predicting how much refinement will improve action quality requires assessing refined actions before they are generated. Proposal uncertainty alone does not establish whether the refiner can improve a proposal or how much correction is worthwhile, while evaluating alternative refinements online consumes the computation we seek to save. Second, offline demonstrations provide expert actions, but no direct labels for refinement needs or the benefit of each allocation. The router must therefore learn from demonstrations to predict action errors at different refinement levels, including no refinement. These predictions allow it to decide whether and how much to refine by weighing the expected reduction in error against the additional computation.

We introduce RAIN (Refine Actions If Needed) to address these challenges through offline comparative supervision and pre-refinement risk prediction. Here, risk refers to the predicted deviation of an action chunk from expert actions. At demonstration states, frozen proposal and refiner models generate candidates whose deviations from the same expert action chunk provide continuous surrogate targets for a risk ensemble. Offline calibration balances predicted risk, uncertainty, computation, and temporal consistency. The router then selects direct execution or only the chosen refinement, without online reward optimization or refinement probes. We instantiate this framework with Action Chunking with Transformers (ACT) [5] as the proposal policy and flow matching as the refiner.

We evaluate the framework on eight ManiSkill tasks [6] and four real-robot tasks. In simulation, our policy achieves 66.6% success at 0.322 ms/action of amortized policy latency, approaching the best evaluated diffusion-policy success rate. On the real robot, it improves success over ACT by 17.5 percentage points with comparable average successful completion time. Matched controls further show that dynamic routing maintains the strongest observed fixed-refinement success level while reducing policy latency by 15.7%.

The paper makes three contributions:

- RAIN, an adaptive action-refinement framework that decides whether and how much to refine an executable action proposal before invoking a generative refiner, preserving a direct-execution path.
- A demonstration-only learning procedure that predicts continuous candidate risks from expert-action deviations, with offline calibration of uncertainty, computation cost, and temporal consistency.
- Evaluation on eight ManiSkill tasks and four real-robot tasks demonstrating the success–efficiency benefits of executable-proposal refinement, with matched controls isolating the additional computation savings from dynamic routing.

II. Related Work

Action policies and generative efficiency. Imitation policies model robot actions through direct prediction, implicit models, or tokenized representations. ACT predicts executable action chunks in one forward pass [5], while implicit behavior cloning and behavior transformers offer alternative representations of complex action distributions [7], [8], [9]. Diffusion and flow-matching policies instead construct action sequences through iterative denoising or vector-field integration [1], [2], [10]. This expressive generation process requires repeated network evaluations at each control decision, motivating efforts to reduce inference cost.

Several approaches reduce this cost by shortening generation or reusing computation. Consistency Policy, One-Step Diffusion Policy, and FlowPolicy enable few-step or single-step prediction through distillation or consistency training [11], [12], [13]. Warm-start methods exploit temporal continuity or predict informative initial actions [14], [15], [16], while streaming generation and improved flow couplings offer complementary efficiency gains [17], [18], [19]. However, distillation can trade action accuracy or diversity for speed [17], while temporal reuse alone may not ensure compatibility with the current state [16]. These trade-offs motivate starting from an executable proposal and allocating correction according to its predicted value.

Executable proposals and learned refinement. Informative starting points offer another route to efficient generation. BRIDGeR refines source-policy samples [3], while A2A and Vision-to-Action (VITA) construct flow sources from action history and visual representations [4], [20]. These methods show that generation need not begin from unstructured noise, but an informative source alone does not determine whether further refinement justifies its cost. When the source is already executable, refinement becomes optional. RAIN explicitly compares the predicted value and cost of correction against direct execution, making the proposal both a refinement starting point and an executable alternative.

Learning corrections to an existing policy is also central to residual-policy methods. Residual reinforcement learning, FISH, and ResiP improve base policies through environment interaction [21], [22], [23], while TRANSIC obtains corrective supervision from human interventions during deployment [24]. These approaches improve existing behavior but require additional interaction or intervention data for correction learning. RAIN instead learns refinement and its allocation from offline demonstrations, using candidate deviations from expert actions to supervise relative correction value. This supports adaptive refinement without collecting online rewards or human corrections.

Adaptive computation for generative policies. Adaptive computation allocates different generation effort to different control decisions. AdaFlow adjusts integration using conditional variance linked to discretization error [25], while ProbeFlow estimates trajectory complexity from initial and lookahead velocity vectors [26]. Prefix-Optimal Generative Policies (POGP) instead learn intermediate denoising values

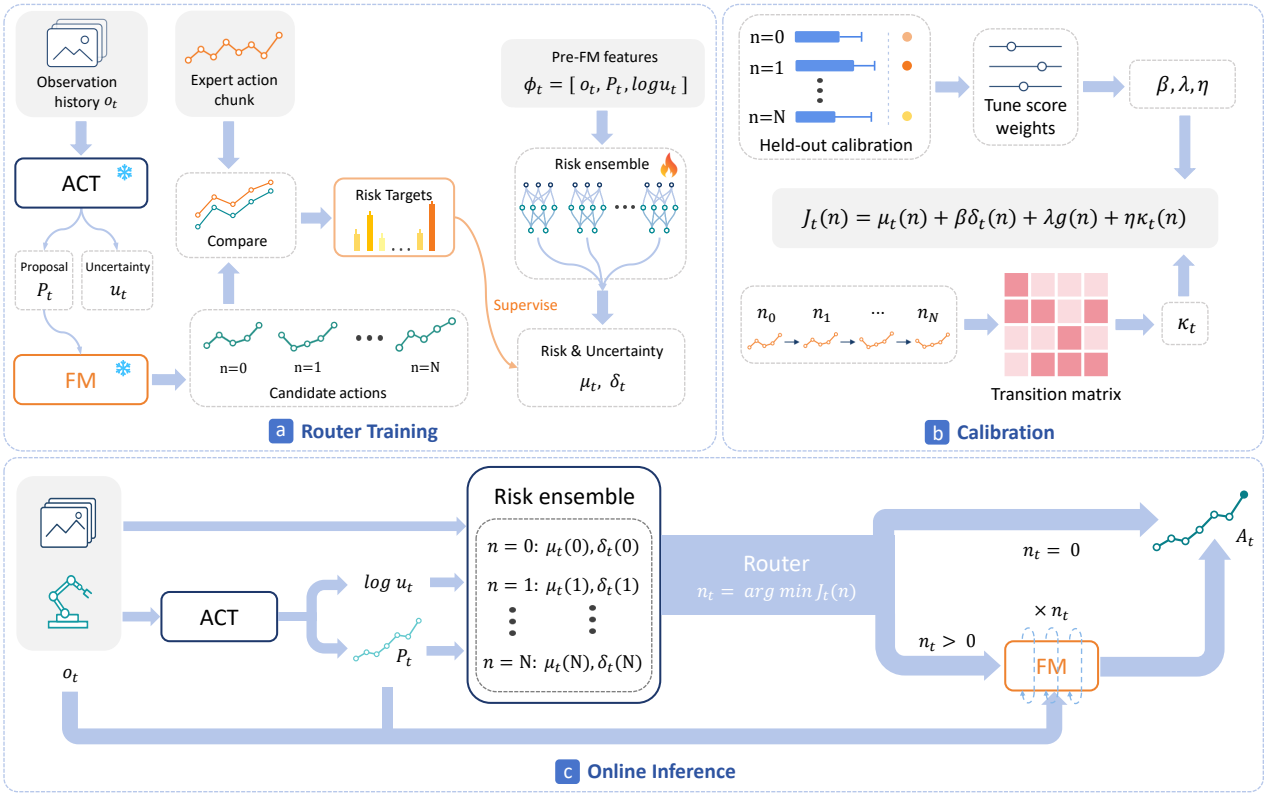


Fig. 2. Overview of RAIN and its risk-routing mechanism. Expert-action deviations of unrefined and refined candidates supervise a risk ensemble; held-out trajectories calibrate uncertainty, compute, and temporal consistency; online, one pre-FM decision either executes the proposal or applies only the selected correction.

to stop when further generation offers little benefit [27]. These criteria adapt computation within generation but do not directly compare refinement against an independently executable proposal. RAIN makes this comparison before the first refiner evaluation, allowing direct execution when the predicted correction benefit does not justify its cost.

Other approaches learn computation allocation from observations. Dynamic Denoising Diffusion Policy (D3P) jointly trains a step adaptor and diffusion policy through reinforcement learning [28], whereas RAIN learns refinement allocation from offline demonstrations. Difficulty-Aware Stochastic Interpolant Policy (DA-SIP) selects integration steps and solver configurations, while the Vision-Adaptive Diffusion Policy Framework (VADF) adjusts denoising schedules and execution lengths [29], [30]. These methods adapt generation or execution; RAIN instead compares correction amounts for the same executable proposal, including no correction. Modality-Adaptive Robot Sampling (MARS) selectively invokes stochastic generation when behavioral diversity is needed [31]. Yet even a single-modal action can require precise correction. RAIN therefore predicts continuous candidate risks, preserving correction differences that can be weighed against uncertainty and computation cost.

III. Method

As shown in Fig. 2, RAIN starts from an executable action proposal and treats generative refinement as optional correction. We first train the proposal policy and refiner, then freeze them. Using offline demonstrations, we compare actions at different refinement levels, including no refinement, with expert actions to obtain error targets. We train an ensemble to predict these errors before running the refiner. At deployment, these predictions guide whether and how much to refine the proposal, and only the selected refinement amount is executed.

A. Executable Proposals and Optional Refinement

At each planning step t, the proposal policy uses the available observation history $\mathbf{o}_t$ to produce an executable action chunk $\mathbf{P}_t \in \mathbb{R}^{H \times d_a}$ and a scalar uncertainty signal $u_t > 0$. Here, H is the chunk length and d_a is the action dimension. In our ACT implementation, $\mathbf{P}_t$ is constructed deterministically from mean action predictions, including temporal aggregation where applicable. The uncertainty summarizes predicted arm-action variance; it informs routing but is never sampled into the proposal.

The refinement amount n specifies the number of refiner evaluations, with $n = 0$ retaining direct execution:

$$\mathbf{A}_t^{(n)} = \begin{cases} \mathbf{P}_t, & n = 0, \\ \pi_{\mathrm{FM}}^{(n)}(\mathbf{P}_t \mid \mathbf{o}_t), & n > 0. \end{cases} \tag{1}$$

Flow matching starts from the proposal rather than noise. Positive refinement amounts integrate the learned velocity field to correct the arm-action trajectory, while retaining the

proposal's gripper commands. All candidates start from the same proposal and have the same action horizon. The router decides which candidate to use from information available before the first refiner evaluation.

B. Joint Proposal–Refiner Training

The proposal should remain useful both for direct execution and as a starting point for correction. We therefore retain its supervised action-prediction objective while jointly optimizing the refiner.

For an aligned expert action chunk $\mathbf{A}_t^E$, flow matching [2] interpolates between the proposal and expert actions. Using normalized arm-action components in the following expression, its training objective is

$$\begin{aligned} \mathbf{X}_\tau &= (1-\tau)\mathbf{P}_t + \tau \mathbf{A}_t^E, \\ \mathcal{L}_{\mathrm{FM}} &= \mathbb{E}\Big[\big\| v_\psi(\mathbf{X}_\tau, \tau \mid \mathbf{o}_t, \mathbf{P}_t) \\ &\qquad - (\mathbf{A}_t^E - \mathbf{P}_t)\big\|_{\mathrm{valid}}^2\Big], \end{aligned} \tag{2}$$

where $\tau \sim \mathcal{U}[0,1]$, and the masked squared error averages over valid action entries. The velocity field thus learns how to move a proposal toward the demonstrated action sequence.

The joint objective is

$$\mathcal{L}_{\mathrm{joint}} = \frac{\mathcal{L}_{\mathrm{FM}}}{\bar{L}_{\mathrm{FM}}} + \alpha_{\mathrm{ACT}} \frac{\mathcal{L}_{\mathrm{ACT}}}{\bar{L}_{\mathrm{ACT}}}, \tag{3}$$

where the reference magnitudes are fixed before joint optimization. The ACT loss trains the proposal policy to predict actions and their variance. The flow-matching loss also updates the proposal policy, encouraging it to produce action chunks that the refiner can improve. Joint training thus encourages proposals that are both directly executable and easier to refine. Both models are subsequently frozen for router training.

C. Offline Candidate-Risk Supervision

To learn when correction is useful, we evaluate alternative refinement amounts at demonstration states. Each candidate is compared with the same aligned expert chunk:

$$\begin{aligned} \ell_t(n) &= d_\gamma(\mathbf{A}_t^{(n)}, \mathbf{A}_t^E), \\ y_t(n) &= \log\left(1 + \frac{\ell_t(n)}{\ell_{\mathrm{ref}}}\right). \end{aligned} \tag{4}$$

Here, d_γ is a masked, dimension-normalized arm-action error with temporal weights γ^h, and $\ell_{\mathrm{ref}} > 0$ is a shared training-set scale. The logarithmic transformation reduces the influence of large errors while preserving candidate ordering. All candidates, including direct execution, receive continuous targets.

Expert-action deviation provides dense supervision for comparing alternatives without branching the environment or collecting rollout rewards. A smaller error means that the candidate action chunk is closer to the expert demonstration. Comparing these errors across refinement levels helps determine whether additional refinement improves the proposal.

D. Pre-Refinement Risk Prediction

To predict both action errors and the uncertainty of these predictions, we use an ensemble of M independently initialized networks [32]. Each network receives observation features, the executable proposal, and $\log u_t$. Each member predicts a mean and log variance for every candidate's transformed error. Training combines heteroscedastic regression with a Huber term and auxiliary pairwise-ranking and best-candidate objectives. The continuous targets retain information about candidate differences instead of reducing supervision to a single preferred refinement label.

After reversing target standardization, the predicted mean and log variance on the transformed-error scale are $\hat{y}_t^{(k)}(n)$ and $\hat{\rho}_t^{(k)}(n)$, respectively. We aggregate them as

$$\begin{aligned} \mu_t(n) &= \max\Big(0, \mathbb{E}_k[\hat{y}_t^{(k)}(n)]\Big), \\ \delta_t(n) &= \sqrt{\mathbb{E}_k[e^{\hat{\rho}_t^{(k)}(n)}] + \mathrm{Var}_k[\hat{y}_t^{(k)}(n)]}. \end{aligned} \tag{5}$$

Thus, $\mu_t(n)$ estimates transformed candidate error, while $\delta_t(n)$ combines predicted variance and ensemble disagreement. Both are learned for direct execution as well as refined candidates. Proposal uncertainty u_t is an input to this prediction; it is distinct from the ensemble's uncertainty $\delta_t(n)$ and does not impose a separate refinement threshold.

E. Offline Calibration and Online Routing

At deployment, the router balances predicted error against uncertainty, computation, and temporal consistency:

$$\begin{aligned} J_t(n) &= \mu_t(n) + \beta\delta_t(n) + \lambda g(n) + \eta\kappa_t(n), \\ n_t &= \arg\min_n J_t(n). \end{aligned} \tag{6}$$

The nondecreasing cost $g(n)$, with $g(0) = 0$, represents relative refinement computation. A refined candidate is selected only when its combined score is preferable to direct execution; a larger refinement amount is not assumed to be better.

The temporal term captures how the refinement amount with the lowest expert-action error changes along offline demonstration trajectories, $n_t^\star = \arg\min_n \ell_t(n)$. We count transitions between these amounts to estimate a smoothed transition probability matrix $\mathbf{T}$. Let n_{t^-} be the refinement amount chosen at the previous planning step, and let Δ_t be the number of environment steps since that decision. Then,

$$\kappa_t(n) = -\log\big(\max\big\{[\mathbf{T}^{\Delta_t}]_{n_{t^-},n}, \epsilon\big\}\big). \tag{7}$$

The first decision has zero transition cost. This soft prior favors plausible refinement sequences without excluding any candidate.

We calibrate β, λ, η on offline demonstration trajectories, with the data allocation specified in Sec. IV. Settings are evaluated by their trajectory-averaged disagreement with the minimum-error candidate. Among settings whose excess error over the best setting is within one paired standard error, we select the one with the lowest mean refinement amount. The coefficients are then frozen before closed-loop evaluation.

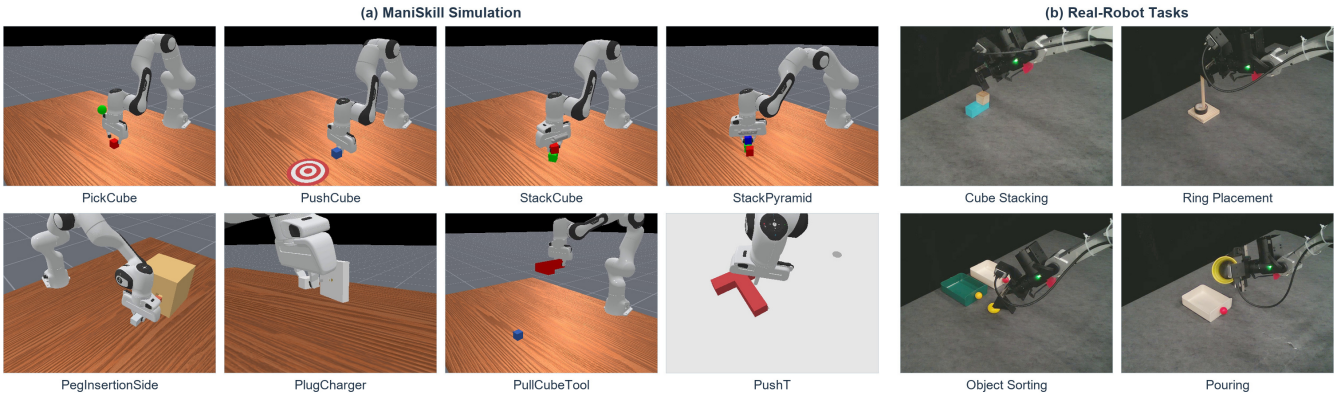


Fig. 3. Task overview. (a) Eight ManiSkill3 manipulation tasks. (b) Four real-robot tasks: Cube Stacking, Ring Placement, Object Sorting, and Pouring. Real-robot images have been brightness-adjusted for visibility.

Online, the ensemble predicts all candidate scores without running the refiner. The policy either executes $\mathbf{P}_t$ directly or constructs only the selected refined candidate. Alternative candidates are generated for offline supervision, not for online search.

IV. Experiments

We evaluate overall performance, dynamic versus fixed refinement, risk prediction versus direct refinement prediction, the transition prior, and task-level refinement allocation.

A. Experimental Setup

Tasks and data. We evaluate on eight ManiSkill3 tasks [6] and four real-robot tasks shown in Fig. 3. Simulation policies use state observations, while real-robot policies use two RGB camera views and proprioceptive observations. Each simulated task provides 500 successful demonstrations. All simulation baselines are trained on all 500 demonstrations, while our framework uses 400 for training, 50 for calibration, and 50 held out from fitting and selection. Real-robot experiments use a six-degree-of-freedom ARX arm with a gripper. Each real task provides 50 training demonstrations recorded at 30 Hz. Each method is evaluated on 100 shared environment seeds per simulated task and 20 trials per real task.

Compared methods. We compare RAIN with ACT, A2A, DP, FM, and ProbeFlow in simulation, and with ACT and DP on the robot. Numeric suffixes denote iterative network evaluations. We show A2A with 1, 4, and 6 evaluations. The simulation ACT baseline uses the official ACT checkpoint with an eight-action execution horizon. For ProbeFlow, we apply its adaptive solver to the frozen FM baseline without retraining, retaining Gaussian initialization. We use a probe time of 0.5, a score threshold of 0.008, and integration schedules of 2, 4, 6, 8, or 10 steps; latency includes both probe evaluations. For the matched simulation study, Fixed 0–3, Direct-C, and RAIN share frozen proposal/refiner checkpoints, preprocessing, evaluation seeds, and an eight-action execution block. Fixed k applies k refinements at each policy call, whereas RAIN selects the amount dynamically. Fixed 0 directly executes our jointly trained proposal, rather than the official ACT checkpoint. Simulation routers are fit and calibrated on the simulation splits. Real-robot router fitting and calibration reuse all 50 training demonstrations per task, without a separate calibration split.

Metrics. We report per-task and overall success rates separately for simulation and real-robot experiments. Overall success is calculated across 800 simulated episodes or 80 real trials per method. Simulated episodes succeed when the environment first reports completion; real trials succeed when the task is completed. Simulation policy latency measures isolated 32-bit floating-point (FP32) CUDA-Graph inference on an otherwise idle NVIDIA A800, from observation availability to action-chunk readiness, excluding environment execution and host-to-device transfer. Latency and FM evaluations are reported per executed action. Real-robot task completion time is measured with policy inference on an NVIDIA RTX 4080 and includes observation processing, policy inference, and physical execution. We average completion time over successful trials within each task and then equally across tasks; unsuccessful trials contribute only to success rate.

B. Overall Success Rate and Efficiency

Q1. Can proposal refinement combine high success rates with efficient execution?

ManiSkill results. Table I and Fig. 4 show that RAIN achieves a 66.6% success rate at 0.322 ms/action, the highest success rate among all evaluated methods below 1 ms/action. We examine this operating point from two perspectives: the success gained over low-cost policies and the computation saved relative to iterative generative policies.

Compared with ACT, RAIN improves success from 41.4% to 66.6% with approximately 0.100 ms/action of additional policy latency. Compared with the highest-success A2A setting, which generates actions from an action-history representation, RAIN gains 11.3 percentage points for only 0.062 ms/action of additional latency. These comparisons show that our proposal-refinement system achieves substantially higher success while remaining close to these policies in inference cost.

TABLE I

SUCCESS RATE AND POLICY LATENCY ON EIGHT MANISKILL TASKS. BOLD INDICATES THE BEST RESULT IN EACH COLUMN. RANK 1 IS BEST; TIES SHARE A RANK, WITH SUBSEQUENT RANKS SKIPPED.

Method	Pick	Push	Stack	Peg	Plug	Pull	PushT	Pyramid	Overall success (%) ↑	Rank	Policy latency (ms/action) ↓	Rank
ACT	98	95	5	7	4	84	3	35	41.375	13	0.222	3
A2A-1	99	50	86	1	0	11	12	3	32.750	14	**0.127**	**1**
A2A-4	99	**100**	90	8	10	31	**29**	36	50.375	11	0.206	2
A2A-6	**100**	**100**	94	27	7	33	**29**	53	55.375	10	0.260	4
DP-1	0	0	0	0	0	4	0	0	0.500	15	0.378	7
DP-2	**100**	99	95	40	7	74	21	32	58.500	8	0.746	8
DP-5	**100**	**100**	**98**	48	**22**	**99**	20	56	**67.875**	**1**	1.839	12
DP-10	**100**	**100**	97	56	14	**99**	15	59	67.500	2	3.120	14
DP-100	**100**	99	**98**	55	17	98	21	52	67.500	2	38.517	15
FM-1	**100**	98	85	12	3	25	16	22	45.125	12	0.376	6
FM-2	**100**	**100**	95	32	2	76	17	42	58.000	9	0.747	9
FM-5	**100**	**100**	94	42	5	93	17	50	62.625	6	1.835	11
FM-10	**100**	**100**	**98**	44	9	96	15	51	64.125	5	3.074	13
ProbeFlow	**100**	99	96	37	3	78	16	49	59.750	7	0.931	10
RAIN	**100**	96	91	**59**	11	89	24	**63**	66.625	4	0.322	5

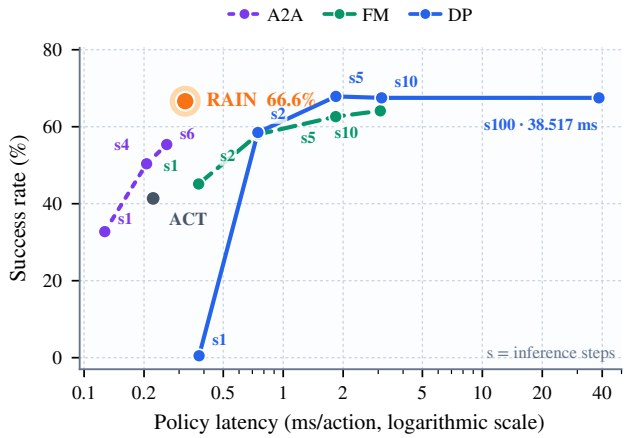


Fig. 4. Success rate versus policy latency on a logarithmic latency axis. Colors and markers identify method families; numeric labels denote the number of network evaluations in each setting.

TABLE II

SUCCESS RATE AND TASK COMPLETION TIME ON FOUR REAL-ROBOT TASKS.

Method	Success Rate (%) ↑					Time (s) ↓
	Stack	Ring	Sort	Pour	Overall	
ACT	65	50	40	65	55	**17.17**
DP	70	50	55	**70**	61.25	21.49
RAIN	**85**	**65**	**70**	**70**	**72.5**	17.57

Stack: Cube Stacking; Ring: Ring Placement; Sort: Object Sorting; Pour: Pouring. Time is averaged over successful trials within each task, then equally across tasks.

Relative to iterative generative policies, the main advantage is retaining high success with substantially less computation. RAIN surpasses the FM setting with the highest success rate by 2.5 percentage points and reduces policy latency from 3.074 to 0.322 ms/action, a 9.55× reduction. It also approaches the highest-success DP setting, which achieves 67.9%, while reducing latency from 1.839 to 0.322 ms/action, a 5.71× reduction. This advantage extends to the adaptive ProbeFlow baseline: RAIN raises success from 59.8% to 66.6% while reducing latency from 0.931 to 0.322 ms/action, a 2.89× reduction. Thus, the evaluated proposal-refinement system improves both success and latency over the strongest FM setting and ProbeFlow, while approaching the best DP success rate at substantially lower latency.

Real-robot results. We next examine whether these benefits carry over to real-robot manipulation, where efficiency is measured by successful task completion time rather than isolated policy latency. RAIN achieves the highest overall success rate across the four tasks, reaching 72.5%, compared with 55.0% for ACT and 61.25% for DP (Table II). It outperforms ACT on every task and matches or exceeds DP's success rate. This higher success is accompanied by an average successful completion time close to ACT's (17.57 versus 17.17 s) and shorter than DP's (21.49 s). The real-robot results therefore extend the simulation findings: executable-proposal refinement supports higher task success without a substantial increase in average successful completion time relative to ACT.

These comparisons establish the performance of the complete system; the following matched controls isolate the additional contribution of dynamic routing.

C. Matched Routing Controls

Q2. Can dynamic refinement improve success while reducing inference cost relative to fixed refinement?

To isolate refinement allocation itself, we compare RAIN with Fixed 0–3 under the matched protocol in Table III. All variants share the frozen proposal and refiner, preprocessing, evaluation seeds, eight-action execution horizon, and accelerated inference implementation; only the allocation of FM refinement across policy calls differs. This comparison separates the benefit of refinement from the additional benefit of deciding when and how much to refine.

Fixed 0 also outperforms the standalone ACT baseline, possibly because the additional variance-prediction objective helps the proposal policy learn task-relevant action variability.

TABLE III

SUCCESS RATE AND INFERENCE COST OF MATCHED ROUTING VARIANTS.

Method	Success (%) ↑	Latency (ms/action) ↓	FM evals. (per action) ↓
Fixed allocation			
Fixed 0	46.5	**0.158**	**0.000**
Fixed 1	65.1	0.278	0.125
Fixed 2	65.8	0.330	0.251
Fixed 3	66.4	0.382	0.376
Direct refinement prediction			
Direct-C Calibrated	65.1	0.310	0.265
Direct-C Argmax	65.9	0.362	0.335
Transition-prior ablation			
RAIN w/o Transition Prior	66.4	0.325	0.288
RAIN	**66.6**	0.322	0.276

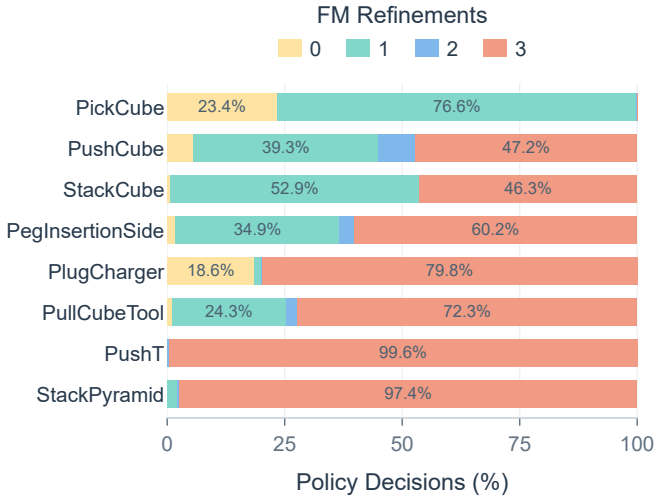


Fig. 5. Task-level refinement allocation by RAIN. Each stacked bar shows the percentage of policy decisions using each refinement amount in our ManiSkill implementation, with each bar totaling 100%.

Dynamic allocation achieves a slightly higher success rate with lower policy latency than the strongest fixed-refinement setting. RAIN reaches 66.6% success at 0.322 ms/action, compared with 66.4% at 0.382 ms/action for Fixed 3, reducing latency by 15.7% and FM evaluations per action by 26.6%. These results demonstrate a better success–latency operating point through state-dependent refinement.

RAIN also improves over Fixed 2, increasing success from 65.8% to 66.6% while reducing latency from 0.330 to 0.322 ms/action, despite using slightly more FM evaluations.

D. Continuous Risk versus Direct Refinement Prediction

Q3. Does predicting candidate risks improve the success–compute trade-off over directly predicting refinement?

To isolate the router supervision target, we compare RAIN with two direct refinement predictors under the matched protocol in Table III. All variants use identical pre-refinement inputs and three-member ensembles. Direct-C converts each offline state into a hard label identifying the refinement amount with the lowest expert-action deviation, whereas RAIN predicts a numerical action-error estimate for each refinement level, referred to here as continuous risk. Direct-C Argmax deploys the raw class prediction, while Direct-C Calibrated applies offline uncertainty, compute, and transition calibration.

Calibration improves the efficiency of direct prediction, reducing latency from 0.362 to 0.310 ms/action and FM evaluations per action from 0.335 to 0.265, with success changing from 65.9% to 65.1%. Continuous risk prediction offers a stronger success-oriented operating point: RAIN achieves 66.6% success at 0.322 ms/action, improving success while reducing both latency and refinement evaluations relative to Argmax. Compared with calibrated Direct-C, RAIN gains 1.5 percentage points of success for only 0.012 ms/action of additional latency.

These results support predicting action errors for each refinement level rather than only predicting which level to choose. A single label does not distinguish between cases where refinement levels have similar errors and those where one has a much lower error. Predicting the error at each level preserves these differences, allowing the router to weigh the expected reduction in action error against the additional computation.

E. Effect of the Transition Prior

Q4. Does the transition prior improve refinement efficiency?

With the proposal, refiner, and risk predictions held fixed, adding the transition prior reduces FM evaluations per action from 0.288 to 0.276 and policy latency from 0.325 to 0.322 ms/action, while maintaining a comparable success rate of 66.6% versus 66.4% without the prior.

The prior complements current-state risk estimates with refinement-transition patterns learned from offline demonstrations. It favors plausible refinement sequences without excluding any candidate, allowing the current observation and proposal to continue determining the allocation. The ablation supports using this temporal information to further improve refinement efficiency, complementing the main allocation capability provided by candidate-risk prediction.

F. How the Router Allocates Computation

Q5. How does RAIN allocate refinement across tasks?

Figure 5 reveals distinct refinement profiles across tasks. PickCube relies almost entirely on zero or one refinement, while PushCube, StackCube, and PegInsertionSide use broader mixtures. PushT and StackPyramid, in contrast, use the highest refinement amount in 99.6% and 97.4% of decisions, respectively. The router therefore produces selective rather than uniformly reduced computation: it retains high refinement on some tasks while exploiting direct execution and lighter correction on others.

Together with the matched controls, these distributions explain how RAIN reduces computation without imposing a lower fixed refinement amount everywhere. Fixed 3 incurs the highest refinement cost at every decision, whereas RAIN selectively avoids that cost while maintaining a comparable overall success rate. The value of dynamic routing lies in

this selective allocation, not in forcing balanced use of all refinement amounts or frequent switching.

G. Refinement Beyond Flow Matching

The main experiments instantiate generative refinement with flow matching. To examine whether executable proposals can also support diffusion-based correction, we consider an auxiliary ACT–diffusion experiment. Within this separate setting, refinement improves success from 52.9% for the unrefined proposal to 63.6% at the best evaluated fixed-refinement setting. These results provide supporting evidence that the proposal-refinement principle extends beyond flow matching; the dynamic routing results in this paper remain evaluated with the flow-matching refiner.

V. Conclusion and Limitations

Conclusion. We presented RAIN, a framework for adaptive generative refinement of executable action proposals. A computationally lightweight policy provides an action chunk, while a router learned from offline demonstrations decides whether and how much to refine it before invoking the generative model. Experiments on eight ManiSkill tasks and four real-robot tasks demonstrate high success rates at low simulation policy latency and improved real-robot success with successful task completion times comparable to ACT. Matched simulation controls further show that dynamic routing achieves a slightly higher success rate with lower latency than the strongest fixed-refinement setting. By starting from executable action proposals and dynamically adjusting refinement to the current environment, RAIN achieves high task success rates while avoiding unnecessary generative computation.

Limitations. The router learns from action errors relative to expert demonstrations, which may not fully reflect the effect of refinement on task success, especially in states outside the demonstration distribution. Learning refinement decisions that better predict task outcomes remains an important direction.

References

[1] C. Chi, *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Int. Conf. Learn. Represent. (ICLR)*, 2023.

[3] K. Chen, E. Lim, K. Lin, Y. Chen, and H. Soh, "Don't start from scratch: Behavioral refinement via interpolant-based policy diffusion," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[4] J. Jia, *et al.*, "Action-to-action flow matching," in *Proc. Robot.: Sci. Syst. (RSS)*, 2026.

[5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.

[6] S. Tao, *et al.*, "Demonstrating GPU parallelized robot simulation and rendering for generalizable embodied AI with ManiSkill3," in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.

[7] P. Florence, *et al.*, "Implicit behavioral cloning," in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2022, pp. 158–168.

[8] N. M. Shafiullah, Z. Cui, A. Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 22 955–22 968.

[9] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, "Behavior generation with latent actions," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 26 991–27 008.

[10] E. Chisari, N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada, "Learning robotic manipulation policies from point clouds with conditional flow matching," in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 982–993.

[11] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, "Consistency policy: Accelerated visuomotor policies via consistency distillation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[12] Z. Wang, *et al.*, "One-step diffusion policy: Fast visuomotor policies via diffusion distillation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 63 399–63 416.

[13] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu, "FlowPolicy: Enabling fast and robust 3D flow-based policy via consistency flow matching for robot manipulation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 39, no. 14, 2025, pp. 14 754–14 762.

[14] H. Chen, *et al.*, "FALCON: Fast visuomotor policies via partial denoising," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 8552–8573.

[15] Y. Duan, H. Yin, and D. Kragic, "Real-time iteration scheme for diffusion policy," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 11 758–11 764.

[16] J. Li, *et al.*, "STEP: Warm-started visuomotor policies with spatiotemporal consistency prediction," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2026.

[17] S. H. Høeg, Y. Du, and O. Egeland, "Fast policy synthesis with variable noise diffusion models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 4821–4828.

[18] S. Jiang, X. Fang, N. Roy, T. Lozano-Pérez, L. P. Kaelbling, and S. Ancha, "Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 238–257.

[19] A. Sochopoulos, N. Malkin, N. Tsagkas, J. Moura, M. Gienger, and S. Vijayakumar, "Fast flow-based visuomotor policies via conditional optimal transport couplings," in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 3357–3377.

[20] D. Gao, *et al.*, "VITA: Vision-to-action flow matching policy," in *Int. Conf. Learn. Represent. (ICLR)*, 2026.

[21] T. Johannink, *et al.*, "Residual reinforcement learning for robot control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6023–6029.

[22] S. Haldar, J. Pari, A. Rai, and L. Pinto, "Teach a robot to FISH: Versatile imitation from one minute of demonstrations," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.

[23] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, "From imitation to refinement - residual RL for precise assembly," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 1–8.

[24] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, "TRANSIC: Sim-to-real policy transfer by learning from online correction," in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 1691–1729.

[25] X. Hu, B. Liu, X. Liu, and Q. Liu, "AdaFlow: Imitation learning with variance-adaptive flow-based policies," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 138 836–138 858.

[26] Z. Fang, J. Wang, Y. Zhou, and Q. Shi, "ProbeFlow: Training-free adaptive flow matching for vision-language-action models," *arXiv preprint arXiv:2603.17850*, 2026.

[27] R. K. Salla, M. Saravanan, and S. Stepputtis, "Learning when to stop: Prefix-optimal dynamic diffusion policies for continuous control," *arXiv preprint arXiv:2608.05084*, 2026.

[28] S.-A. Yu, F. Gao, Y. Wu, C. Yu, and Y. Wang, "D3P: Dynamic denoising diffusion policy via reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2026, accepted.

[29] I. Chun, S. Lee, M. Albergo, S. Xie, and E. Vanden-Eijnden, "Dynamic test-time compute scaling in control policy: Difficulty-aware stochastic interpolant policy," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, pp. 51 441–51 462.

[30] X. Yu, Z. Liu, S. Nan, S. Wu, and Y. Fu, "VADF: Vision-adaptive diffusion policy framework for efficient robotic manipulation," *arXiv preprint arXiv:2604.15938*, 2026.

[31] J. Jia, *et al.*, "MARS policy: Multimodality only when it matters," *arXiv preprint arXiv:2605.29766*, 2026.

[32] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.